

i# Functional Programming Exam: Monday, May 19, 2025

- The exam duration is five hours.
- There are four questions. To obtain full marks you must answer all the subquestions satisfactorily.
- You are allowed to use books, lecture notes, lecture slides, hand-ins, solutions to assignments, calculators, computers, software etc. during the examination. This includes F# compilers and IDEs.
- You may **NOT** use the internet in any way other than WiseFlow for the duration of the exam and you must be connected to WiseFlow, and its monitoring software, at all times.
- You may **NOT** use any form of code generators like Visual Studio CoPilot, ChatGPT or equivalent. All code you hand in must either be in the template we provide, or written by you yourself. The use of any such tool is grounds for disciplinary action. Standard auto-completion like Intellisense is ok.
- You may **NOT** communicate with anyone else during the exam, which includes using chat clients, phones, hand gestures, smoke signals, etc. The exception is the invigilators if you have a question.
- All forms of software that can be used to communicate, like Steam, Messenger, Discord, Teams, etc, and any code generation tools (CoPilot, LLMs like ChatGPT, etc) must be completely switched off for the entire exam. It's not enough not to use them, they must be switched off completely. To be safe, restart your computer before the exam and make sure that no banned programs are running.
- You are (unless otherwise instructed) allowed to use the .NET library including the modules described in the book, e.g., List, Set, Map etc.
- If a question requires you to define a function, then you may (unless otherwise instructed)
 - use that function in subsequent subquestions, even if you have not managed to define it. Providing the signature of the missing function will help in such cases.
 - define as many helper functions as you want, but in any case you must define the required function so that it has exactly the type and effect, and the helper functions must follow the same constraints, that the question asks for. A tail recursive assignment may, for instance, be coded as a non recursive main function that calls a tail recursive auxiliary function.
- Unless explicitly stated you must provide functional solutions without side effects
- You are required to use the provided code project `FPEXam2025` as a basis for your submission and you should **only hand in** the `Exam.fs` file. No other file, and **absolutely not a zip file of the entire project**. The project includes everything you need to run as an independent project. Any helper functions that we provide in the `Exam.fs` file may also be part of your submission.
- If an assignment asks that you write a function `isEven : int -> bool`, for instance, then there is a corresponding `let isEven _ = failwith "not implemented"` in `Exam.fs`. You may change these functions (changing a `let` to a `let rec` for instance) as long as their signatures match those given in the assignment. In this case that could be `let isEven x = x % 2 = 0`.

You MUST include explanations and comments to support your solutions for the questions that require them. You simply write them as comments around your code.

Your entire exam hand-in MUST be made by yourself and yourself alone It is illegal to make the exam answers as group work or to enlist the help of others in any way. This includes using solutions or code found online, or tools that write code for you (such as CoPilot or ChatGPT).

Your solution MUST compile. We reserve the right to fail any submission that does not meet this minimal requirement.

1: Coordinates and rectangles (25%)

Consider the two following types `coord` and `rectangle`

```
type coord    = C of int * int
type rectangle = R of coord * int * int
```

Where a coordinate $C(x, y)$ represents a coordinate in two-dimensional space with the x-coordinate x and the y-coordinate y , and a rectangle $R(c, w, h)$ represents a rectangle with its lower left corner at coordinate c , the width w and the height h . You may assume that

- the x-coordinates increase left-to-right
- the y-coordinates increase bottom-to-top
like a standard coordinate system

Question 1.1

A rectangle is valid if both its height and its width are strictly greater than zero.

Create a function `valid` of type `rectangle -> bool` that given a rectangle r returns `true` if the r is a valid rectangle, and `false` otherwise.

Examples:

```
> valid (R (C (-1, 4), 2, 3))
val it: bool = true

> valid (R (C (-1, 4), 2, 0))
val it: bool = false

> valid (R (C (-1, 4), -3, 3))
val it: bool = false
```

Question 1.2

Create a recursive, but not tail-recursive, function `coords` of type `rectangle -> Set<coord>` that given a valid rectangle r returns all of the coordinates that lie within the rectangle. You **may not** use list- or set comprehension.

You may assume that r is valid. You do not have to check for it. Also, you may have a recursive (not tail-recursive) auxiliary function do the main work and have `coords` call that.

Examples:

```
> coords (R (C (-1, 4), 1, 1))
val it: Set<coord> = set [C (-1, 4)]

> coords (R (C (-1, 4), 2, 3))
val it: Set<coord> =
  set [C (-1, 4); C (-1, 5); C (-1, 6); C (0, 4); C (0, 5); C (0, 6)]
```

Question 1.3

Create a tail-recursive function `coordsAcc` using an accumulator (not a continuation) that behaves exactly the same as `coords` for all possible valid inputs.

Questions 1.4

For these questions you may only use the following functions, but you do not have to use all of them.

- Higher-order functions from the `List` library
- Functions from the `Set` library
- Function composition (`>>`) and/or piping (`|>`)
- The functions `valid`, `coords`, and `coordAcc` from Questions 1.1, 1.2, and 1.3 respectively.

Create the non-recursive functions `merge` of type `list rectangle -> Set<coord>` where given a list of rectangles `rs` returns the set of all coordinates of all of the valid rectangles in `rs`.

Hint: The `Set` library contains useful functions for doing unions on sets.

Examples:

```
> merge [R (C (-1, 4), 1, 2); R (C (2, 4), -1, 2); R (C (-1, 4), 2, 1)]  
val it: Set<coord> = set [C (-1, 4); C (-1, 5); C (0, 4)]
```

Question 1.5

For this exercise you may not use any of the functions that you have created so far, nor any comprehension functions (list or set comprehension for instance), nor any functions from the standard library. You may also not create auxiliary functions that mimic any of these functions.

Create a higher-order function `rectFold` of type `('b -> coord -> 'b) -> 'b -> rectangle -> 'b` that given a folding function `f`, an initial accumulator `acc` and a rectangle `r`, with corner coordinate `C(x, y)`, width `w` and height `h`, returns

```
f (... f (f (... (f acc C(x, y)) ...)
          (C(x + w - 1, y + h - 2))
          (C(x, y + h - 1)) ...)
  (C(x + w - 1, y + h - 1))
```

In other words, the folding function loops over all coordinates in the `r` row by row, i.e. first by increasing `x`-values until the end of the row is reached, then by increasing the `y`-value by one and setting the `x` value to the start of the row, and continuing until all rows have been parsed. For example,

```
rectFold f acc (R(C(0, 0), 2, 2)) =
  f (f (f (f acc (C(0, 0)))
        (C(0, 1)))
    (C(1, 0)))
  (C(1, 1))
```

Create a non-recursive function `coords2` that behaves exactly like `coords` and `coordsAcc` for all possible inputs, but that is constructed using `rectFold`.

Examples:

```
> rectFold (fun (xs, ys) (C(x, y)) -> (x :: xs, y :: ys)) ([], []) (R(C(0, 0), 2, 2))
val it: int list * int list = ([1; 0; 1; 0], [1; 1; 0; 0])

> coords2 (R (C (-1, 4), 2, 3))
val it: Set<coord> =
  set [C (-1, 4); C (-1, 5); C (-1, 6); C (0, 4); C (0, 5); C (0, 6)]
```

2: Code comprehension (25%)

Consider the following functions

```
let rec foo a b =  
  match a, b with  
  | a :: c, b :: d when a < b -> a :: foo c (b :: d)  
  | a :: c, b :: d             -> b :: foo (a :: c) d  
  | [], _ -> b  
  | _, [] -> a  
  
let rec bar a =  
  match List.length a with  
  | 0 -> [] (* 1 *)  
  | 1 -> a  (* 2 *)  
  | l -> foo (bar a.[0..l / 2 - 1]) (bar a.[l / 2 .. l - 1]) (* 3 *)
```

Question 2.1

- What do the functions `foo` and `bar` do? Focus on what they do rather than how they do it.
- What would be appropriate names for functions `foo` and `bar`?
- The function `foo` only behaves reasonably if certain constraint(s) are met on its argument. What is/are these constraints?

Question 2.2

Consider the match statement in `bar`. What would happen if you

- Swap lines `(* 1 *)` and `(* 2 *)`
- Swap lines `(* 1 *)` and `(* 3 *)`
- Swap lines `(* 2 *)` and `(* 3 *)`

You must justify your answer.

Question 2.3

The line

```
match List.length a with
```

in the `bar` function causes the function to not be as performant as it could be. Explain why.

Create a function `bar2` that behaves exactly the same as `bar` for all possible inputs but that does not have this performance problem. It may be useful to construct an auxiliary function to help with this.

You may not use any functions from the standard library other than `List.length` nor any form of comprehension functions, like list comprehension.

Question 2.4

The function `foo` from Question 2.1 is not tail recursive. Explain why. To make a compelling argument you must evaluate a function call of the function, similarly to what is done in Chapter 1.4 of HR, and reason about that evaluation. You need to make clear what aspects of the evaluation tell you that the function is not tail recursive. Keep in mind that all steps in an evaluation chain must evaluate to the same value ($(5 + 4) * 3 \rightarrow 9 * 3 \rightarrow 27$, for instance).

Question 2.5

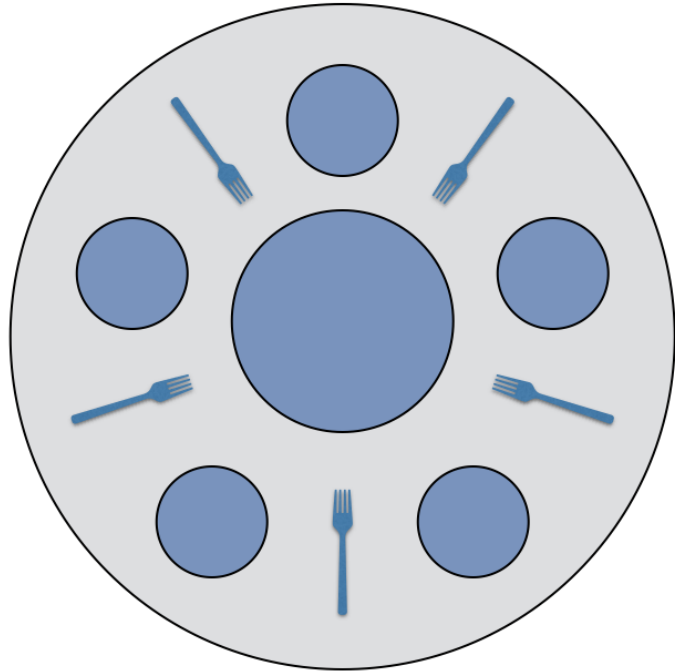
Create a function `fooTail` that behaves the same way as `foo` for all possible inputs but which is tail recursive and coded using continuations.

3: Dining philosophers (25%)

The dining philosophers problem is a classical problem in computer science where a group of ill-fated philosophers sit around a table with a bowl of food (the type of which varies) but with only one fork each, placed between each pair of philosophers

Each philosopher will either grab both forks on their either side and eat for a bit, or sit and ponder about the world's problems. One such problem arises if two adjacent philosophers try to eat at the same time - if all philosophers grab one fork they will never get to eat as their other fork will never be available.

We will slowly work our way up to solving this problem.



Question 3.1

Source: OpenCSF

We represent the table as an array of booleans

```
type table = T of bool []
```

where each index represents a fork and the value is `true` if the fork is picked up and `false` if the fork is on the table. The philosophers can be thought as sitting between the indexes. This means that philosopher number 4 will have his left fork at index 4 and his right fork at index 5. The table is circular so if the table has size 10 then philosopher 9 (we start counting at zero as usual) will have his left fork at index 9 and his right fork at index 0.

Important: For this assignment (3.1 only) you may update the elements of the array directly even though they are mutable.

Remember that we use the following commands

- `arr.[i]` to get element at index `i` from an array `arr`
- `arr.[i] <- x` to update the element at index `i` of array `arr` to `x`
- There are many ways to create an array, but we suggest you use `Array.create`

Create a function `setTable` of type `int -> table` that given an integer `size` creates a table of size `size` where all of the forks are on the table.

Create the function `getLeftFork` of type `table -> int -> unit` that given a table `t` and a philosopher `p` has the philosopher `p` try to pick up their left fork by setting its value to `true` in the `table` array. If the value is already `true` then print The philosopher `<pid>` tried to pick up their left fork, but it is already taken to the terminal, where `<pid>` is the philosopher `p`.

Create the function `getRightFork` that is identical to `getLeftFork` but where the philosopher picks up the right fork in stead, and the error message states that they pick up their right fork rather than their left. Remember to wrap around the array for the final philosopher and be careful of precedences -- $a + b \% c$ is equal to $a + (b \% c)$ and not $(a + b) \% c$, where the latter is most likely what you will want to use.

Create the functions `putLeftFork` and `putRightFork` that behave the same as `getLeftFork` and `getRightFork` respectively but where

- the forks are put down rather than taken up
- The error message The philosopher `<pid>` tried to put down their `<left/right>` fork, but it is already on the table is printed to the terminal if a fork is already on the table, and where `<left/right>` is left for `putLeftFork` and right for `putRightFork`.

Warning: Make sure these functions are correct. They can cause very hard-to-catch bugs.

Also note that none of these functions check that the philosopher who picks up a fork also is the one to put it down. This would just over complicate things. It is possible for philosopher 0 to pick up their right fork only for philosopher 1 to put it down as their left.

Examples:

```
> let t = setTable 5
val t: table = T [|false; false; false; false; false|]

> getRightFork t 0; t
val it: table = T [|false; true; false; false; false|]

> getLeftFork t 0; t
val it: table = T [|true; true; false; false; false|]

> getLeftFork t 0; t
Philosopher 0 tried to pick up their left fork, but it is already taken
val it: table = T [|true; true; false; false; false|]

> putLeftFork t 0; t
val it: table = T [|false; true; false; false; false|]

> getRightFork t 4; t
val it: table = T [|true; true; false; false; false|]

> putRightFork t 4; t
val it: table = T [|false; true; false; false; false|]

> putRightFork t 4; t
Philosopher 4 tried to put down their right fork, but it is already on the table
val it: table = T [|false; true; false; false; false|]
```


Question 3.2

You must use the functions from Q3.1 for this question. You may not update the array using any other functions. To make sure this doesn't happen by accident, never match on the `table` type to get at the array under the `T` constructor.

Create the function `eat` of type `table -> int -> unit` that given a table `t` and a philosopher `p` has the philosopher eat by picking up their left and their right fork from the table `t` using `getLeftFork` and `getRightFork`.

Create the function `think` of type `table -> int -> unit` that given a table `t` and a philosopher `p` has the philosopher think by placing their left and their right fork on the table `t` using `putLeftFork` and `putRightFork`.

Create the function `canEat` of type `table -> int -> bool` that given a table `t` and a philosopher `p` returns `true` if both left and right forks of philosopher `p` are on the table `t` and `false` otherwise.

Examples:

```
> let t = setTable 5
val t: table = T [|false; false; false; false; false|]

> eat t 4; t
val it: table = T [|true; false; false; false; true|]

> eat t 1; t
val it: table = T [|true; true; true; false; true|]

> eat t 0; t
Philosopher 0 tried to pick up their left fork, but it is already taken
Philosopher 0 tried to pick up their right fork, but it is already taken
val it: table = T [|true; true; true; false; true|]

> eat t 2; t
Philosopher 2 tried to pick up their left fork, but it is already taken
val it: table = T [|true; true; true; true; true|]

> think t 1; t
val it: table = T [|true; false; false; true; true|]

> think t 1; t
Philosopher 1 tried to put down their left fork, but it is already on the table
Philosopher 1 tried to put down their right fork, but it is already on the table
val it: table = T [|true; false; false; true; true|]

> canEat t 1
val it: bool = true

> canEat t 0
val it: bool = false
```

Question 3.3

Important: This is the hardest question in Q3. You can still get full credit for Q3.4 and Q3.5 even if you do not solve this one at all. You will need the `message` and `philosopherTable` types as well as the `inbox` function defined below. Regardless, we highly recommend you read both 3.3 and 3.4 as they do tie together.

We will now make sure that philosophers do not deadlock the system, by going for the same forks at once, by using a mailbox. At no point from now on should you see the error messages from Q3.1 be printed. If they are, you have a bug. The mailbox should make sure that two philosophers cannot go for the same forks at the same time.

The intuition is the following:

- The mailbox defined here contains the table as defined in Q3.1 and will update this table using the functions `eat` and `think` defined in Q3.2.
- The mailbox will receive messages when a philosopher (defined in Q3.4) wants to eat or think. That is the philosophers request to be allowed to eat or think, but only the mailbox picks up or puts down their forks.
- When a philosopher wants to eat, and the forks on both sides are available, the mailbox will pick up the forks (using `eat`) and send a message back to the philosopher that they can start eating.
- When a philosopher is done eating and wants to think the mailbox will put the forks down (using `think`) without sending a message back.

Use the following code skeleton

```
type message =  
  | Eat of int * AsyncReplyChannel<unit>  
  | Think of int  
type philosopherTable = Phil of MailboxProcessor<message>  
  
let inbox size (mbox : MailboxProcessor<message>) =  
  let t = setTable size  
  
  let messageLoop (pending : (int * AsyncReplyChannel<unit>) list) =  
    failwith "not implemented"  
  
  messageLoop []
```

This mailbox contains a table `t` as described in Q3.1 and the `messageLoop` function which receives messages from the philosophers and updates this table.

The type `message` allows the philosopher to eat or think. The message

- `Eat(p, rc)` informs the mailbox that philosopher `p` would like to eat and that it should return a unit message `()` on the return channel `rc` when the philosopher can eat (when their forks on both sides have been picked up by the mailbox).
- `Think(p)` informs the mailbox that philosopher `p` is done eating and would like to think.

In the `inbox` function, a table `t` of the correct size is initialised. This table does not have to be passed around as an argument but will be mutated by the `messageLoop` function, using the `eat` and `think` functions you created in 3.2.

The main loop `messageLoop` takes a list of pairs as an argument where each pair contains the id of a philosopher, and their return channel. The list represents the philosophers who want to eat but currently cannot because their neighbors are

currently eating.

The `messageLoop` function itself should use the functions you created in 3.2 and :

- Receive the next message from the inbox
- If the message has the form `Eat(p, rc)` then
 - if the forks on both sides of philosopher `p` are available then
 - pick up their forks
 - send a unit `()` back on `rc` to inform them that they holding the forks and can start eating
 - print philosopher `<p>` is eating to the terminal, where `<p>` is the philosopher identity `p`.
 - recurse with the same `pending` list since the philosopher `p` is allowed to eat
 - If a fork on either side of philosopher `p` is unavailable then recurse and add `(p, rc)` to the head of the `pending` list. The philosopher cannot eat now, but must wait until the forks are available.
- If the message has the form `Think(p)` then
 - Let the philosopher think (place their forks down)
 - print philosopher `<p>` is thinking to the terminal, where `<p>` is the philosopher identity `p`.
 - Go through the `pending` list and allow any philosopher that can eat to do so in the same way as if they had been allowed to eat when they first requested to do so
 - Recurse with the `pending` list without the philosophers who were allowed to eat.

Note that we never check if a philosopher is allowed to think (if both forks are put down) it is up to the client of the mailbox (the philosopher) to not attempt to do this.

Examples: You will find examples in Q3.4

Question 3.4

Create a function `newTable` of type `int -> philosopherTable` that given a size `size` initialises a new mailbox and returns it as a `philosopherTable`.

Create a function `philEat` of type `philosopherTable -> int -> unit` that given a philosopher's table `pt` and a philosopher `p` sends a message to `pt` that the philosopher `p` would like to eat, the function returns once `pt` has sent a message back along a return channel.

Create a function `philThink` of type `philosopherTable -> int -> unit` that given a philosopher's table `pt` and a philosopher `p` sends a message to `pt` that the philosopher `p` is done eating and would like to think. The function terminates immediately and does not wait for an answer from `pt`.

Examples: In `Program.fs` you will find two functions, `testQ34_block` and `testQ34_succeed`, where the first one should block and the second one should succeed.

Running `testQ34_block` should give you

```
philosopher 0 is eating  
philosopher 2 is eating  
philosopher 0 is thinking
```

and then it should block.

Running `testQ34_succeed` should give you

```
philosopher 0 is eating  
philosopher 2 is eating  
philosopher 2 is thinking  
philosopher 0 is thinking  
philosopher 1 is eating  
Everyone is done eating (well, philosopher one is still eating)
```

Question 3.5

For this question you may use `Async.Parallel` even though it returns an array. You may read from this array, but you may not modify it in any way.

We will use a random number generator to determine how long a philosopher eats and thinks.

The following random number generator lets you generate numbers.

```
let random = System.Random(42)
```

Moreover, the function

- `random.Next` has type `int -> int` and given a number `x` generates a random number between `0` and `x - 1`.
- `Async.Sleep` has type `int -> Async<unit>` and given a number `t` sleeps the current process for `t` ms, while the rest of the processes keep running.

We will now let every philosopher run asynchronously from each other, where each philosopher eats a certain number of times from the table, and if the `philosopherTable` mailbox is working correctly, they will not deadlock.

Create a function `philosopher` of type `int -> int -> int -> philosopherTable -> Async<unit>` that given a philosopher id `p`, a number of times that the philosopher will eat `meals`, a maximum time `time`, in ms, that the philosopher will take to eat or think, and a philosopher's table that the philosopher is sitting by `pt`, does the following `meals` times

- Tries to eat
- Once they are allowed to eat they eat randomly between `0` and `time` ms.
- Thinks between `0` and `time` ms

Once the philosopher has eaten `meals` times the function terminates.

Create a function `diningPhilosophers` of type `int -> int -> int -> unit` that given a number of philosophers `phils`, the number of times they should eat `meals`, and the time in ms that they take to eat and think `time`, the function creates a `philosopherTable` of size `phils`, spawns `phils` philosophers sitting around the table and runs them all in parallel. When it is done it should print `Everyone is done eating` to the terminal.

Examples: The function `runDiningPhilosophers` in `Program.fs` runs five philosophers eating two times each, using the command `let`

```
runDiningPhilosophers () = diningPhilosophers 5 2 1000
```

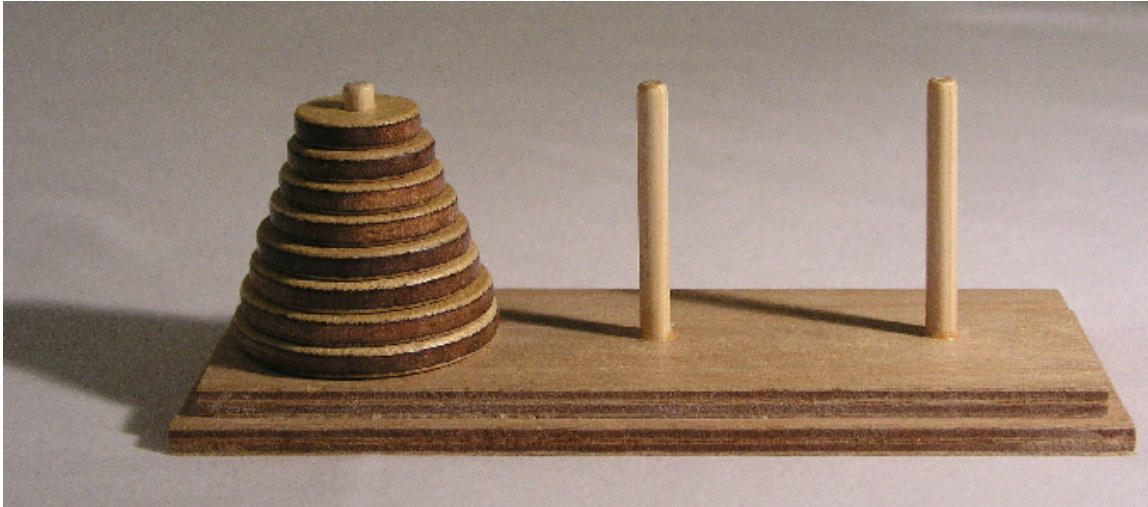
and each eating and thinking for one second. A sample output is the following

```
philosopher 1 is eating
philosopher 4 is eating
philosopher 4 is thinking
philosopher 3 is eating
philosopher 1 is thinking
philosopher 0 is eating
philosopher 3 is thinking
philosopher 2 is eating
philosopher 0 is thinking
philosopher 4 is eating
philosopher 2 is thinking
philosopher 1 is eating
philosopher 1 is thinking
philosopher 2 is eating
philosopher 4 is thinking
philosopher 0 is eating
philosopher 2 is thinking
philosopher 3 is eating
philosopher 3 is thinking
philosopher 0 is thinking
Everyone is done eating
```

Your order may be different, but the function should terminate, and the eating order should be correct (philosopher 0 and 1 cannot eat at the same time for instance).

4: The Towers of Hanoi (25%)

The Towers of Hanoi is an ancient puzzle where a set of discs of decreasing size are placed on a structure with three pegs. The object of the game is to move the tower from the leftmost peg (the starting peg) to the rightmost peg (the goal peg) with the help of the middle peg without ever placing a larger disc on top of a smaller one.



Source: Wikipedia

The main part of this assignment is finding a good representation of this game in `F#` and coding some elementary functions like moving one disc from one peg to another.

Question 4.1

Create a type `peg` that represents one of the pegs on the board.

Create the values `start`, `middle`, and `goal` all of type `peg` to represent the starting peg, the middle peg, and the goal peg respectively.

Create a type `disc` to represent the discs in the game. It must be possible to determine the size of a disc. Don't overcomplicate this.

Create a function `size` of type `disc -> int` that given a disc `d` returns the size of the disc.

Create the type `hanoi` to represent a current state of the game. It must be able to tell what discs are on what pegs.

There are many correct answers to what is a good representation of a game, so don't think too much about if you have the perfect one, but it should be easy to use with the functions that you create below, so it is a good idea to read the rest of the question before you commit to a design.

Create a function `newGame` of type `int -> hanoi` that given an integer `numDiscs` returns a `hanoi` game where `numDiscs` discs of decreasing size, placed from bottom to top, are placed on the starting peg, and where the middle and goal pegs are empty. We will call this starting state the *initial configuration* of the game.

We will use the following notation to describe a board

```
4 | 1
5 | 2
-----
```

where, in this case, discs of size

- 5 and 4 are placed on the starting peg
- 3 is placed on the middle peg
- 2 and 1 are placed on the goal peg

To increase readability, we also use | to represent an empty part of a tower.

Examples:

```
> newGame 3

val it : hanoi = 
  1 | | 
  2 | | 
  3 | | 
  -----
```

Create a function `isFinished` of type `hanoi -> bool` that given a game `h` returns `true` if `h` is a finished game, where the goal peg contains a tower of decreasing disc size, (bottom to top), where the size difference is at most 1 and the smallest disc has size 1.

Examples:

```
> isFinished 
  1 | | 
  2 | | 
  3 | | 
  -----
val it : bool = false

> isFinished 
  | | 1
  | | 2
  | | 3
  -----
val it : bool = true

> isFinished 
  | | 1
  | | 2
  | | 4
  -----
val it : bool = false
```


Question 4.2

For this question we will be using the `Result` type to report the following types of errors.

```
type error =  
  | Empty of peg  
  | Invalid of peg * disc * disc
```

Where

- `Empty p` means that we tried to take a disc from an empty peg `p`
- `Invalid(p, d1, d2)` whenever we try to place the disc `d1` on top of a disc `d2` on the peg `p` when the size of `d1` is greater the size of `d2` (we are not allowed to put a bigger disc on top of a smaller one).

Create a function `take` of type `peg -> hanoi -> Result<disc * hanoi, error>` that given a peg `p` and a hanoi game `h` returns

- `Ok(d, h')` where `d` is the disc at the top of the tower of peg `p` in `h` and `h'` is the same as `h` but with `d` removed from `p`
- `Error(Empty p)` if there are no discs on the peg `p` in `h`

Create a function `place` of type `peg -> disc -> hanoi -> Result<hanoi, error>` that given a peg `p`, a disc `d`, and a hanoi game `h` returns

- `Ok h'` where `h'` is `h` but with the disc `d` placed on top of peg `p`, if the size of `d` is not greater than the top disc on `p`
- `Error(Invalid(p, d, d'))`, where `d'` is the disc on top of `p` that is smaller than `d` otherwise

Question 4.3

For this assignment we will be using a state monad to hide the hanoi game. The state monad you will be working on is very similar to the one that you used for the yellow assignments in your interpreter, but the state is much simpler (the hanoi game from Question 4.1).

```
type hanoiMonad<'a> = HM of (hanoi -> Result<'a * hanoi, error>)  
  
let ret x = HM (fun h -> (Ok (x, h)))  
let fail err = HM (fun _ -> Error err)  
let bind f (HM a) =  
  HM (fun h ->  
    match a h with  
    | Ok (x, h') ->  
      let (HM g) = f x  
      g h'  
    | Error err -> Error err)  
  
let (>=) a f = bind f a  
let (>>=) a b = a >= (fun _ -> b)  
  
let evalHM (HM f) l = f (newGame l)
```

Note that `evalHM` evaluates the `hanoiMonad` from an initial configuration of the game.

Create functions `take2` of type `peg -> hanoiMonad<disc>` and `place2` of type `peg -> disc -> hanoiMonad<unit>` where

- `take2`, given a peg `p`, takes the top disc from peg `p` in the hanoi game and fails if this is not possible
- `place2`, given a peg `p` and a disc `d`, places the disc `d` on top of peg `p` in the hanoi game and fails if this is not possible.

These functions should be implemented using `take` and `place` from Q4.2 respectively, and successful results, or errors, should be returned in the state monad as appropriate. Moreover failure, means monadic fail (using `fail`) and not `failwith`.

Important: You cannot use monadic operators, like `bind` or `ret`, for `take2` or `place2` as you must break the abstraction of the hanoi monad to implement these functions, but we include them here for debugging purposes, and you will need them for Question 4.4.

Examples:

```
> take2 start |>
  evalHM 3 |>
  Result.map (fst >> size)
val it: Result<int,error> = Ok 1

> take2 start >=>
  place2 middle >>=>
  take2 start |>
  evalHM 3 |>
  Result.map (fst >> size)
val it: Result<int,error> = Ok 2

> take2 goal |>
  evalHM 3 |>
  Result.map (fst >> size)
val it: Result<int,error> = Error (Empty <your representation of goal>)

> take2 start >=>
  place2 middle >>=>
  take2 start >=>
  place2 middle |>
  evalHM 3 |>
  Result.map (fst >> size)
val it: Result<unit,error> =
  Error (Invalid (<your representation of middle>,
    <your representation of a disc of size 2>,
    <your representation of a disc of size 1>))
```

Question 4.4

For this assignment you may, if you want to, use computation expressions in which case you will need the following definitions.

```
type HanoiBuilder() =
    member this.Bind(f, x)    = bind x f
    member this.Return(x)    = ret x
    member this.ReturnFrom(x) = x
    member this.Combine(a, b) = a >>= (fun _ -> b)

let han = new HanoiBuilder()
```

You may also solve the assignment using monadic operators, but you may not break the abstraction of the state monad (you may not pattern match on anything of type `hanoiMonad`).

Create a function `move` of type `peg -> peg -> hanoiMonad<unit>` that given a `peg from` and a `peg to` moves the top disc from `peg from` to `peg to` and fails monadically (using `fail` from Q4.3 and not `failwith`) if an illegal move is performed.

Create a function `doMoves` of type `(peg * peg) list -> hanoiMonad<unit>` that given a list of moves `moves`, where each move is given as a pair containing a source and a destination `peg` (the `peg` that the disc is moving from to the `peg` that the disc is moving to), performs all moves in `moves` using the `move` function and fails monadically if an illegal move is performed. The monadic operators from Q4.3 (`>>=` and `ret`) should suffice.

To test your function you may use the following function that solves the towers of hanoi.

```
let rec solveHanoi size from via dest =  
  match size with  
  | 0 -> []  
  | x -> solveHanoi (x - 1) from dest via @  
          (from, dest) ::  
          solveHanoi (x - 1) via from dest
```

Understanding it is not a part of this exam, but the intuition is that it moves a tower of size `size` from a `from` peg via a `via` peg to a destination peg `dest` by first moving `size - 1` discs from the `from` peg to the `via` peg via the `dest` peg, then moving the remaining largest disc from the `from` peg to the `dest` peg, and then finally moving the `size - 1` discs on the `via` peg to the `dest` peg via the `from` peg. It's a really clean (and optimal) recursive solution and solving it with loops is much more complicated.

Examples:

```
> solveHanoi 3 start middle dest |>  
  doMoves |> evalHM 3 |>  
  Result.map (snd >> isFinished)  
val it: Result<bool,error> = Ok true  
  
> move start middle >>=  
  (solveHanoi 3 start middle dest |> doMoves) |>  
  evalHM 3 |>  
  Result.map (snd >> isFinished)  
val it: Result<unit,error> =  
  Error (Invalid (<your representation of middle>,  
                  <your representation of a disc of size 3>,  
                  <your representation of a disc of size 1>))
```

Question 4.5

Create the parses `pstart`, `pmiddle`, and `pgoal` of type `Parser<peg>` that parses the strings `"start"`, `"middle"`, and `"goal"` to the pegs `start`, `middle`, and `goal` respectively.

Create a parser `parsePeg` of type `Parser<peg>` that either parses the start, the middle, or the goal peg.

Create a function `parseMove` of type `Parser<peg * peg>` that returns a parser for a string of the form `"<peg to be parsed by parsePeg>-><peg to be parsed by parsePeg>;"`. Pegs are separated by `"->"` and moves are terminated by `";"`. There are no spaces.

Create a function `parseMoves` of type `Parser<(peg * peg) list>` that given a, possibly empty, sequence of moves, as described in `parseMoves` returns the list of those moves.

Examples:

```
> run parseMoves "start->middle;start->goal;middle->goal;"
val it: Result<(peg * peg) list,string> = Ok [(start, middle); (start, goal); (middle, goal)]

> run parseMoves "start->middle;start->goal;middle->start;"
val it: Result<(peg * peg) list,string> = Ok [(start, middle); (start, goal)]

> run parseMoves "start>middle;start->goal;middle->start;"
val it: Result<(peg * peg) list,string> = Ok []
```